A Fault-Tolerant Routing Strategy in Hypercube Multicomputers

Ge-Ming Chiu, *Member, IEEE Computer Society*, and Shui-Pao Wu

Abstract—We investigate fault-tolerant routing which aims at finding feasible minimum paths in a faulty hypercube. The concept of unsafe node and its extension are used in our scheme. A set of stringent criteria is proposed to identify the possibly bad candidates for forwarding a message. As a result, the number of such undesirable nodes is reduced without sacrificing the functionality of the mechanism. Furthermore, the notion of degree of unsafeness for classifying the unsafe nodes is introduced to facilitate the design of efficient routing algorithms which rely on having each node keep the states of its nearest neighbors. We show that a feasible path of length no more than the Hamming distance between the source and the destination plus four can always be established by the routing algorithm as long as the hypercube is not fully unsafe. The issue of deadlock freeness is also addressed in this research. More importantly, another fault-tolerant routing algorithm, which requires only a constant of five virtual networks in wormhole routing to ensure the property of deadlock freeness for a hypercube of any size, is presented in this paper.

Index Terms—Deadlock, fault tolerance, hypercubes, routing, virtual channels, wormhole routing.

1 Introduction

Recently, hypercube multicomputers have been drawing considerable attention from many researchers. A large amount of research effort has been directed towards hypercube systems [3], [8], [17], [18]. Due to their regular structure and low diameter, hypercube multicomputers are well suited for parallel processing [11]. Several research and commercial hypercube machines have been built in recent years [10].

In a multicomputer system, efficient communication among the processors is critical to the performance of the system. Processors (or nodes) in a hypercube communicate with each other via passing messages. Hence, routing of messages is an important issue that need to be addressed. In particular, it is essential to design routing strategies that can route messages efficiently in the presence of faulty components. The large number of nodes and communication links in a hypercube intrinsically provides fault-tolerant capability for message routing. If a message path is blocked due to component failures, fault-free alternative paths can be used to forward the message. In this paper, the issue of fault-tolerant routing in hypercube multicomputers is investigated.

A variety of routing algorithms have been proposed for hypercube systems in the past [2], [9], [12]. However, most of these algorithms are not suitable for routing messages when nodes fail in a hypercube. There also has been a number of fault-tolerant routing strategies proposed in the previous research [4], [5], [14], [15], [18], [19], [20]. Gordon and Stout [14] proposed an approach called "sidetracking" to route messages in a hypercube with faulty nodes. A message is derouted to a randomly chosen fault-free neighboring node when there exists no link available for advancing the message. However, routing failures may occur although such probability is low, and excessive message delay may arise.

Chen and Shin [5] developed and analyzed a routing scheme based on depth-first search in which backtracking is required when all forward links are blocked by faulty components. Every message must carry information indicating the dimensions already traversed to avoid routing to the same node except when backtracking is enforced. The time overhead required is relatively excessive when the number of faults is small. A simplified version of the method that tolerates fewer faults was presented in [4]. Most of the foregoing strategies are designed for packet-switching systems only. Furthermore, the issue of deadlock freeness is not specifically considered in these schemes.

Lee and Hayes [15] proposed a different fault-tolerant routing strategy which is based on the concept of unsafe nodes. Message routing is to avoid unsafe nodes which could possibly lead to communication difficulties or excessive delay. It does not require each message to carry information about its past routing history. Instead, each node only has to keep simple information about the states of its neighbors. The computing requirement for the routing algorithm is simplified; the time delay incurred at each node is small. They showed that the algorithm can route a message via a path of length no greater than two plus the Hamming distance between the source and the destination of the message as long as the hypercube is not fully unsafe, which is guaranteed provided that the number of faults is no more than $\lceil n/2 \rceil$ in an n-cube.

In [19], the concept of *safety level*, which is an enhance-

• *G.-M. Chiu is with the Department of Electrical Engineering and Technology, National Taiwan Institute of Technology, Taipei, Taiwan, ROC. E-mail: chiu@optimal.ee.ntit.edu.tw.*
• *S.-P. Wu was with the Department of Electrical Engineering and Technology, National Taiwan Institute of Technology, Taipei, Taiwan, ROC. He is now with AT&T Taiwan, Hsinchu, Taiwan, ROC.*

Manuscript received July 18, 1994; revised Dec. 22, 1994. A preliminary version of this paper appeared in the Proceedings of the 24th International Symposium on Fault-Tolerant Computing, June, 1994.
For information on obtaining reprints of this article, please send e-mail to: transactions@computer.org, and reference IEEECS Log Number C95180.

0018-9340/96$05.00 ©1996 IEEE

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

ment of the safe node concept is presented for implementing reliable broadcasting in hypercubes. The safety level is an approximate measure of the number as well as distribution of faulty nodes. An efficient broadcasting scheme using safety level is proposed. Extensive survey of routing algorithms for hypercube systems can be found in [13], [16].

In this paper, we develop a routing strategy which adopts a similar philosophy, in its nature, to the one proposed in [15]. The concept of unsafe nodes and its extension are used in our scheme. A set of stringent criteria is presented to identify the possibly bad candidates, called unsafe nodes, to forward a message. As a result, the number of such undesirable nodes is reduced without sacrificing the functionality of the mechanism. The same definition of unsafe nodes has also been proposed independently in [20], which specifically addresses the issue of reliable broadcasting in a faulty hypercube. Our paper instead mainly focuses on node-to-node routing. In this paper, we study the properties of the set of unsafe nodes in details. In particular, rigorous proofs are presented. Furthermore, we introduce the notion of degree of unsafeness for classifying the unsafe nodes to facilitate the design of efficient routing algorithms. We show that a feasible path of length no more than the Hamming distance between the source and the destination plus four can always be established by our algorithm as long as the hypercube is not fully unsafe. According to the proposed criteria, this condition is guaranteed if the number of faulty nodes is no more than $n-1$ in an n-cube.

While the issue of deadlock freeness was addressed in some of the previous routing schemes, the property of deadlock freeness is basically achieved by assigning a set of priority classes to communication resources, such as message buffers, and restricting the use of these resources according to a certain order of the classes. One of the main disadvantages of this scheme is that the number of priority classes required for the communication resources increases as the size of the hypercube grows, thereby rendering the design unscalable. In this paper, we propose another fault-tolerant routing algorithm which achieves deadlock freeness with only a constant of five virtual networks for a wormhole-routing hypercube of any size. Deadlock free-ness is ensured by restricting the use of virtual channels in increasing order of network numbers, and having up-transition traversals be considered first over down-transition ones. The notions of up-transition and down-transition links are used in an earlier work for routing in a circuit-switched hypercube [6]. However, they are used in a very different form in this paper. Extension of the results to hypercube systems with link failures is also briefly discussed.

In the next section, necessary notations and definitions are given. The routing strategy and the associated properties are presented in Section 3. The issue of deadlock freeness is addressed in Section 4. In Section 5, we briefly discuss the issue of tolerating link failures. Section 6 draws the conclusion.

2 Preliminaries

An n-dimensional hypercube, abbreviated as n-cube, has $N = 2^n$ processors (or nodes). Each processor T, $0 \le T \le N-1$, can be represented by a sequence of n binary digits $(t_{n-1},\ldots,t_0)$ where $t_i \in \{0, 1\}$ for $0 \le i \le n-1$. The bit t_i is called the i-dimensional bit of node T, and t_0 is the least significant bit. Two processors are connected by a bidirectional link if and only if the binary representations of their labels differ in exactly one bit. A link is said to be an i-dimensional link, $0 \le i \le n-1$, if it connects two nodes that differ in their i-dimensional bits. A path between two nodes can be represented as an ordered sequence of nodes in which these two nodes are the both end nodes and every two consecutive nodes are physically adjacent to each other. Equivalently, this path can be represented as a sequence of communication links connecting the consecutive nodes. The number of links on a path is called the length of the path. The Hamming distance between two nodes X and Y, denoted by $H(X, Y)$, is the number of bits in which the labels of X and Y differ. It is assumed that a faulty node cannot be used for routing messages in a hypercube. A path is called a *feasible* path if it traverses through no faulty nodes. A feasible path is called *minimum* if it has the shortest length among all feasible paths between the source and destination nodes.

3 Fault-Tolerant Routing Strategy

Failed components may block or delay the routing of messages in a hypercube system, possibly degrading system performance. Fault-tolerant routing capability should be exploited by taking advantages of the large amount of nodes and communication links in a hypercube. For instance, there are $r!$ shortest paths between two nodes that are separated by a Hamming distance of r. To illustrate the routing problem, Fig. 1 shows a 4-dimensional hypercube with four faulty nodes, whose addresses are (0010), (0100), (1000), and (1111). Suppose that a message is to be sent from node (1101) to node (0000). In this case, a feasible minimum path of length 3, such as (1101, 1001, 0001, 0000), exists in the system. However, if node (1101) has no knowledge of the failures of nodes (0100) and (1000), the message may first be sent to node (1100). It turns out that there exists no feasible path of length 2 from this intermediate node to the destination, thereby the message cannot be forwarded to its destination through a feasible minimum path of length 3 via node (1100). Now consider another case where a message is to be sent from (0110) to (0000). It is obvious that no feasible minimum path of length 2 exists; thus the message must be derouted. If the message is first derouted

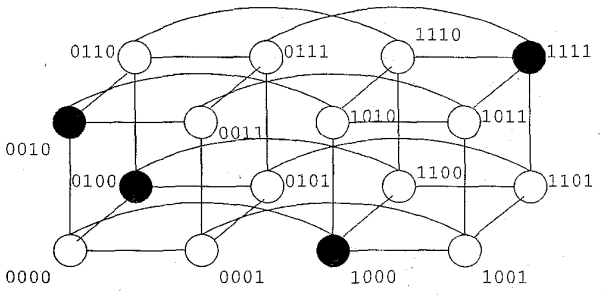


Fig. 1. A 4-dimensional hypercube with four faulty (dark) nodes.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

to node (1110), it will next be forwarded to either of nodes (1010) and (1100), from which no feasible minimum paths of length 2 are possible and another derouting is necessary. This not only increases routing latency but also makes the routing function complex. On the other hand, if the message is first forwarded to node (0111) from (0110), a feasible minimum path of length 4, as opposed to 6, can be readily established.

It would be desirable to route a message through a feasible minimum path in a faulty hypercube. From the previous example, it is apparent that the amount of information, known to each individual node, about the states of other nodes can dictate the likelihood for a distributed routing mechanism to route messages through feasible minimum paths. A natural way of defining the amount of information known to each node is in terms of the range of neighborhood of a node whereby the node has information about the states of all other nodes within this neighborhood [1], [15]. For example, if each node is aware of the state of every other node in an n-cube, i.e., the range of neighborhood associated with the knowledge is n, one can search through all possible routes to locate a feasible minimum path when it sends a message to a given destination. This, however, may incur a significant amount of computation time, and require a considerable size of storage to hold the information.

In the following, we investigate the design of a fault-tolerant routing scheme for hypercubes in which nodes may fail. Each node stores information about the states of its nearest neighbors, which is a common and natural situation. To facilitate our discussion, the consideration of deadlock freeness will not be addressed until Section 4.

3.1 The Unsafe Nodes

In the following, we present a fault-tolerant routing strategy which is based on the idea of avoiding routing a message via a node which may lead to no feasible minimum paths. Both the source and destination nodes of a message are assumed to be fault-free. The notion of an *unsafe* node is introduced to indicate a potentially bad choice for a routing algorithm [15]. Consider a node, say I, which currently has a message destined for a node D, and the Hamming distance $H(I, D)$ is k. If k is equal to 2, there are two node-disjoint minimum paths between these two nodes, and they are the only such paths. It is easy to determine whether a minimum path from node I to node D exists by simply examining if the two nearest neighbors of I which are also nearest neighbors of node D are both faulty. However, if $k \geq 3$, the mere faulty/nonfaulty state information of node I's nearest neighbors is not informative enough in preventing a message from being forwarded to a "bad" next node as illustrated in the previous example. This lays the basis for the following definition.

DEFINITION 1. *A fault-free node is defined as an* unsafe *node if it has either two or more faulty nearest neighbors, or three or more faulty or unsafe nearest neighbors.*

A nonfaulty node that is not unsafe is called a *safe* node. With the definition, the unsafe nodes in a hypercube can be identified in a recursive manner. At each iteration, each node exchanges its current state with all of its fault-free neighbors, and then checks if itself should be marked unsafe with the latest state information. For the ease of discussion, let us assume that the identification function is performed among all fault-free nodes in a synchronous fashion. A simple algorithm for identifying unsafe nodes is shown in Fig. 2. Initially, each node contains a list *FAULT* identifying its faulty neighbors, and a list *SAFE* containing all fault-free neighbors. Firstly, all unsafe nodes that are due to two or more faulty neighbors are identified. It takes three or more unsafe or faulty neighbors to make a nonfaulty node unsafe afterwards. Finally, the unsafe and safe neighbors of a node are kept in *UNSAFE* and *SAFE* lists, respectively. Apparently, the lengths of *FAULT*, *UNSAFE*, and *SAFE* should add up to n for an n-cube. This algorithm is initiated at the system setup time. It is also initiated whenever the status of the system changes, e.g., when a new fault is detected. The status of a node can be changed only when its neighbors change their status. For example, Fig. 3 shows the 4-cube of Fig. 1 with unsafe nodes identified. Node (0110) is unsafe as it has two faulty neighbors, namely (0010) and (0100). By the same token, nodes (1010) and (1100) are also unsafe. Node (1110) will subsequently be marked unsafe due to the aforementioned three unsafe nodes. In Fig. 3, the unsafe nodes are shown with gray filling. Routing strategies should avoid forwarding messages to unsafe nodes as much as possible. Consider the previous example where a message is sent from node (1101) to (0000). By not forwarding the message to node (1100) which is unsafe, a routing function can avoid the routing difficulty

```
Algorithm Iden_Unsafe
{Each node keeps a list FAULT containing its faulty neighbors, a list UNSAFE containing
 its unsafe neighbors, and a list SAFE containing its safe neighbors. Notations l_f and l_u
 are lengths of FAULT and UNSAFE, respectively. Initially, SAFE contains all fault-free
 neighbors and l_f is set accordingly.}
begin
   l_u = 0;
   while (system status is not stable) begin
         for j = 1 to n begin
             Receive status s_X from neighbor X connected by dimension j;
             if (s_X = unsafe and X is not in UNSAFE) then begin
                Add X to UNSAFE;
                Delete X from SAFE;
                l_u := l_u + 1;
             end
         end
         if (l_f ≥ 2 or l_f + l_u ≥ 3) then
            Mark current node as unsafe;
   end
end
```

Fig. 2. The algorithm used to find all unsafe nodes in an n-cube.

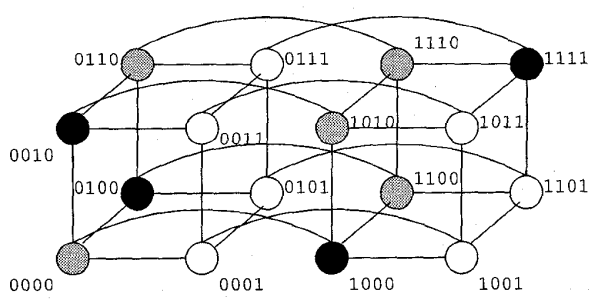


Fig. 3. The unsafe nodes for the 4-cube of Fig. 1.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

described earlier. Instead, it may choose to route via a feasible minimum path such as (1101, 1001, 0001, 0000).

Obviously, a large number of unsafe nodes will reduce the number of alternative paths a message can traverse; thus affecting the flexibility that a routing algorithm can possibly offer. Therefore, a nonfaulty node should be labeled as unsafe only if that is necessary. This should be accompanied by the property that there always exists, between a safe node and any other fault-free node, a feasible path of length equal to the Hamming distance between them.

THEOREM 1. *In a faulty hypercube, if node A is a safe node, for any nonfaulty node B, there always exists a feasible minimum path of length equal to H(A, B) between A and B.*

PROOF. Let $h = H(A, B)$. If h is 1, nodes A and B are nearest neighbors and the link exists. Suppose h is equal to 2. There are two node-disjoint shortest paths from A to B. By definition, node A, which is safe, has at most one faulty neighbor. Consequently, at least one feasible shortest path of length 2 exists. Consider the case where $h \geq 3$. Node A has h nearest neighbors that are on some shortest paths of length h to node B. As node A is safe, the number of unsafe or faulty neighbors of A must be less than three, which means that there exists at least one nearest neighbor, among h of them, of A that is safe. Take any one of these safe neighboring nodes, say node I, as the next node. Obviously, the Hamming distance between I and B will be $h - 1$. Iteratively apply the former procedure to node I and the succeeding nodes as long as the associated distance is greater than or equal to 3. Eventually, one will reach a safe node which is 2 hops from node B. Borrowing the argument for the case of $h = 2$ as described earlier, one has that there exists a shortest partial path from this safe node to node B. Therefore, we have that the full path is feasible and is of length h. Hence, the theorem is proved. □

Theorem 1 provides the ground for the usefulness of safe nodes in a faulty hypercube. It also illustrates the importance of reducing the number of unsafe nodes for routing schemes.

3.2 Properties of the Unsafe Area

Before proceeding, let us study some properties of the structure formed of the safe nodes, or alternatively of the faulty and unsafe nodes. These properties are useful for the design of fault-tolerant routing algorithms.

Let us define *safe set* as the set of safe nodes in a hypercube. It would be interesting to see whether the safe set can be partitioned into disconnected components. A component in this case is a maximal nonempty subset of safe nodes that are connected among themselves.

LEMMA 1. *Suppose that the safe set of an n-dimensional faulty hypercube is not connected. Let SA and SB be any two disconnected components of the safe set. Then there exist two nodes $A \in SA$ and $B \in SB$ such that $H(A, B) = 2$.*

PROOF. Consider a pair of nodes $A \in SA$ and $B \in SB$ such that $H(A, B)$ is minimum among all pairs of nodes in SA and SB. Since SA and SB are disconnected, $H(A, B) > 1$, i.e., A and B cannot be nearest neighbors. Now assume that $l = H(A, B) \geq 3$. There are l nearest neighbors of A on some shortest paths from A to B. As node A is a safe node and $l \geq 3$, at least one of these l nearest neighbors of A is safe, i.e., there exists a safe neighbor, say A', of A such that $H(A', B) = l - 1$. Since nodes A and A' are connected, SA should contain node A' as well. However, this contradicts previous assumption that $l = H(A, B)$ is the shortest distance between the nodes in SA and SB. Hence we show that $H(A, B) = 2$, that is, the minimum distance between two nodes, one each in SA and SB, equals 2. The existence of such nodes is obvious, and the lemma follows. □

Fig. 4 shows an example of a faulty 4-cube in which there are four faulty nodes, whose addresses are (0010), (0100),(1001), and (1111). The safe set of the hypercube contains two disconnected subcubes of dimension 2, namely (0**1) and (1**0), where * is a don't care symbol representing either 0 or 1.

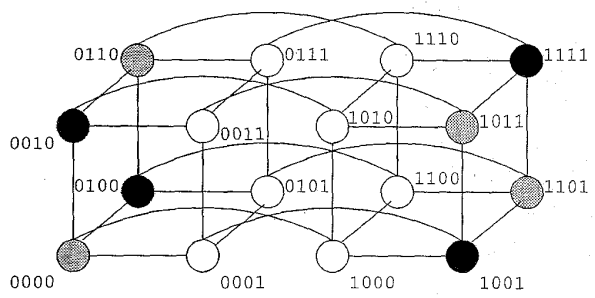


Fig. 4. Faulty 4-cube with disconnected safe set consisted of two subcubes of dimension 2.

THEOREM 2. *For an n-dimensional hypercube with a given set of faulty nodes, if the safe set is not connected, it must be composed of exactly two distinct subsets of safe nodes, where each of them exactly forms a complete $(n - 2)$-dimensional subcube, and they are located in the opposite quarters.*

PROOF. Consider any two distinct subsets SA and SB of safe nodes which correspond to two different components. From Lemma 1, there exist safe nodes $A \in SA$ and $B \in SB$ such that $H(A, B) = 2$. Assume nodes A and B differ in dimensions i and j. Divide the hypercube into four $(n - 2)$-dimensional subcubes along dimensions i and j. Let these four subcubes be denoted by SQ_{00}, SQ_{01}, SQ_{10}, and SQ_{11}, respectively, where the two digits of the subscripts indicate the binary values of the ith and jth bits of the corresponding subcubes. Nodes A and B must be in two opposite subcubes; without loss of generality, assume A is in SQ_{00} and B is in SQ_{11}. Since SA and SB are disconnected, neither of the two common neighbors, A' and B', of A and B is safe; otherwise SA and SB will be connected through the safe one(s). In addition, A' and B' must be contained, separately, in the other two $(n - 2)$-dimensional subcubes. Next we want to prove that SA and SB exactly form the complete SQ_{00}

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

and SQ_{11}, respectively. Now consider a neighboring node A'' of A such that A'' differs from A in dimension k, where $k \neq i$ and $k \neq j$. Apparently, A'' is in SQ_{00}. As node A is safe and it already has two neighbors A' and B' that are both not safe, A'' must be a safe node; thus $A'' \in SA$. Similarly, the neighbor B'' of B along dimension k is also a safe node in SQ_{11} and $B'' \in SB$. Note that the nodes A'' and B'' differ exactly in dimensions i and j. In addition, neither of the two common neighbors of A'' and B'' is safe, and these nodes are contained in SQ_{10} and SQ_{01} separately.

For any node $X \in SQ_{00}$, the addresses of nodes A and X are identical in dimensions i and j. Any shortest path from A to X would contain no link of either dimension i or dimension j, i.e., all the intermediate nodes on the path is in SQ_{00}. Applying the aforementioned argument sequentially to the nodes on any such path, one can conclude that node $X \in SQ_{00}$ is safe. In other words, all the nodes in SQ_{00} are safe. The same is true for SQ_{11}. On the other hand, none of the nodes in SQ_{10} and SQ_{01} can be safe; otherwise the assumption that the safe set is disconnected is violated. Obviously, SQ_{00} and SQ_{11} are located in the opposite quarters of the hypercube. The above argument also rules out the possibility that a safe set can be composed of more than two disconnected components. Hence we prove the theorem. □

Fig. 4 also shows a typical example for Theorem 2. Note that a connected safe set in a hypercube does not necessarily appear in form of a complete subcube as illustrated in Fig. 3.

A faulty hypercube is called *fully unsafe* if the corresponding safe set is empty, i.e., every node in the hypercube is either faulty or unsafe. A fully unsafe hypercube may encounter difficulties in routing messages.

Apparently, the structure of the safe set is dependent on the fault pattern existing in the system. Consider an n-dimensional hypercube in which there are n faulty nodes such that these faults happen to occur at the n neighbors of a nonfaulty node A. Obviously, node A is unsafe. If node B is of distance 2 from A, it has two common neighbors with node A, and these neighbors are both faulty according to the given fault pattern. Hence node B is unsafe. In other words, all the nodes that are of distance 2 from A are unsafe. As to those nodes that are of distance 3 from A, every one of them has three neighbors that are of distance 2 from node A. Since these neighbors are all unsafe, we have that all the nodes that are of distance 3 from A must be unsafe as well. Applying the same argument to other nodes that are of longer distances from A, one can easily conclude that the n-cube is fully unsafe. The following theorem summarizes this observation in a general way.

THEOREM 3. *Given an n-dimensional hypercube, if a node A has k, $2 \leq k \leq n$, faulty neighbors, then every nonfaulty node in the associated k-subcube, which includes node A and the k faulty neighbors, is unsafe.*

In the following we further address the issue on the number of faulty nodes it takes to make a hypercube fully unsafe.

LEMMA 2. *Consider a k-dimensional hypercube with some given set of faulty nodes. If this k-cube is fully unsafe with the fault pattern, the following statement is always true: Divide the k-cube into two $(k-1)$-subcubes, say SQ_0 and SQ_1, along any dimension i, $0 \leq i \leq k-1$. Now construct another independent $(k-1)$-dimensional hypercube such that if either of the two neighboring nodes across dimension i, one each in SQ_0 and SQ_1, is faulty, the corresponding node in the $(k-1)$-cube is also faulty. Then the newly constructed $(k-1)$-cube is fully unsafe by itself.*

PROOF. Let us call the newly constructed $(k-1)$-cube Q. Unsafe nodes in a hypercube can be classified into two categories. Those unsafe nodes having two or more faulty neighbors is called *type 1*. The rest of unsafe nodes belong to *type 2*. Q can be visualized as simply overlapping SQ_0 and SQ_1 and marking a node faulty if either of its corresponding nodes in SQ_0 and SQ_1 is faulty. Now compare Q with SQ_0. If a node is faulty in SQ_0, its corresponding node in Q is faulty too. Consider an unsafe node of type 1 in SQ_0. If the opposite node in SQ_1 is faulty, the corresponding node in Q is obviously faulty. Otherwise, it must have two or more faulty neighbors in SQ_0; hence the corresponding node in Q is unsafe as well. The same argument applies to the comparison between Q and SQ_1. Consequently, the set of faulty or type-1 unsafe nodes in Q contains the union of the corresponding sets of such nodes in SQ_0 and SQ_1. Note that all the unsafe nodes of type 2 are generated iteratively out of the aforementioned set of nodes. Now consider an unsafe node X of type 2 in SQ_0 which is generated immediately as a result of the set of faulty or type-1 unsafe nodes and whose corresponding node in Q is neither faulty nor type-1 unsafe. The opposite node of X, in SQ_1, can be neither faulty nor type-1 unsafe. Hence, there are at least three faulty or type-1 unsafe neighbors of X in SQ_0; thus the corresponding node in Q must also be unsafe. This argument leads to the conclusion that the set of faulty or unsafe, regardless of the type, nodes in Q after each iteration will include the union of the corresponding sets of nodes in SQ_0 and SQ_1. In other words, if a node in SQ_0 or SQ_1 is unsafe as of type 2, its corresponding node in Q is also unsafe. Consequently, the newly constructed $(k-1)$-cube Q is fully unsafe if the original k-cube is fully unsafe. □

The following lemma is also required for upcoming discussion.

LEMMA 3. *If an n-cube with n faulty nodes is fully unsafe, then any two of the n faulty nodes cannot be nearest neighbors.*

PROOF. We prove by induction. For $n = 3$ it is easy to see that the lemma is true. Assume the lemma is true for $n = k$. We want to prove that it is also true for $n = k + 1$. Now, by contradiction, let us assume that the lemma is untrue for $n = k + 1$, i.e., there exists certain case where two of the $k + 1$ faults, say nodes A and B, in a fully unsafe $(k + 1)$-cube are nearest neighbors. Partition the $(k + 1)$-cube, along the dimension in which A and

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

B differ, into two k-subcubes, called SQ_0 and SQ_1, where A is in SQ_0 and B is in SQ_1. Let F_0 and F_1 represent respectively the sets of faulty nodes in SQ_0 and SQ_1, including A and B. Construct a new k-cube Q by overlapping SQ_0 and SQ_1 along with their fault patterns. According to Lemma 2, Q is also fully unsafe. Furthermore, the number of faulty nodes in Q is less than or equal to k as nodes A and B would merge into one single faulty node in Q. Note that, for any pair of nodes $X \in F_0$ and $Y \in F_1$, it must be true that $H(X, Y) \neq 2$. This is due to the reason that if $H(X, Y) = 2$, the two faulty nodes in Q, which correspond to X and Y, would become nearest neighbors; thus contradict the previous assumption for the case of $n = k$. Next, the cardinalities of F_0 and F_1 can be characterized as follows:

$$\min(|F_0|, |F_1|) \leq \left\lfloor \frac{k-1}{2} \right\rfloor + 1 = \left\lfloor \frac{k+1}{2} \right\rfloor = \left\lceil \frac{k}{2} \right\rceil$$

Now assume, without loss of generality, $|F_0| = \min(|F_0|, |F_1|)$. Construct another k-cube Q' in which the fault pattern is exactly identical to that of SQ_0. Note that the number of faults in Q' is no more than $\lceil \frac{k}{2} \rceil$. Hence, if, instead, the criterion used in [15] for labeling a nonfaulty node as unsafe is used in Q', Q' is not fully unsafe as per our proof in [7]. Now let us return to SQ_0. Because the distance between any pair of nodes $X \in F_0$ and $Y \in F_1$ cannot be 2, there is no type-1 unsafe node in SQ_0 which is due to two faulty neighbors, one in F_0 and another in F_1. As a result, the unsafe nodes of type 1 in SQ_0 must be identical to those in Q'. Now consider type-2 unsafe nodes in SQ_0. According to our definition, it takes one more unsafe or faulty neighbor to mark a nonfaulty node type-2 unsafe in SQ_0 than it does in Q'. Therefore, it is easy to see that, if a node is labeled as type-2 unsafe in SQ_0, the corresponding node in Q' must also be marked unsafe after each iteration of labeling. In other words, if a node is safe in Q' the corresponding node in SQ_0 must also be safe. Since Q' is not fully unsafe, the nodes in SQ_0 cannot possibly be fully unsafe. However, this contradicts the assumption that the $(k + 1)$-cube is fully unsafe. Hence we prove the lemma. □

Now we are set to present the following theorem.

THEOREM 4. *If the number of faulty nodes in an n-cube is no greater than $n - 1$, then the n-cube cannot become fully unsafe.*

PROOF. We prove by induction. For $n = 3$ it is easy to see that the theorem is true. Assume the theorem is true for $n = k$. We are to prove that it is also true for $n = k + 1$. Now, by contradiction, let us assume that the theorem is untrue for the case of $n = k + 1$, i.e., some fault pattern with k faults can make the $(k + 1)$-cube fully unsafe. Partition the $(k + 1)$-cube along an arbitrary dimension into two k-subcubes, SQ_0 and SQ_1. The sets of faulty nodes F_0 and F_1 are identically defined as those in the previous lemma. Note that neither F_0 nor F_1 can be empty. For an arbitrary pair of nodes $X \in F_0$ and $Y \in F_1$, X and Y cannot be nearest neighbors, i.e., $H(X, Y) \neq 1$; otherwise one can construct a fully unsafe k-cube with $k - 1$ faulty nodes by overlapping SQ_0 and SQ_1, as X and Y would merge into one single faulty node, and contradicts the case of $n = k$. In addition, if $H(X, Y) = 2$, the two corresponding faulty nodes in the aforementioned fully unsafe k-cube would be nearest neighbors, and this violates Lemma 3. In fact, the foregoing arguments can be applied to any pair of the k faulty nodes. Hence, we have $H(A, B) \geq 3$ for any pair of faulty nodes A and B. Consequently, there is no type-1 unsafe node generated in the $(k + 1)$-cube; therefore the $(k + 1)$-cube cannot be fully unsafe. However, this contradicts the previous assumption that the $(k + 1)$-cube is fully unsafe. Thus the theorem is proved. □

Theorem 4 provides a lower bound on the number of faulty nodes for which a hypercube can become fully unsafe as per our criteria. Although n faults can make an n-cube fully unsafe, it only occurs in very limited cases even when the size of the hypercube is small. Our analysis of many cases shows that fully unsafe situation can happen only when the n faults occur at the n neighbors of some nonfaulty node for $n \geq 6$. In this case, the given nonfaulty node is isolated anyway; routing among other nodes is hardly affected.

3.3 Fault-Tolerant Routing Algorithm

We now use state information of the nodes in a hypercube to assist routing of messages. The routing strategy is based on that each node has knowledge of the states of its nearest neighbors.

Before proceeding, we further classify the unsafe nodes according to the degree of "unsafeness" to facilitate the design of a fault-tolerant routing algorithm. The following definition follows.

DEFINITION 2. *An unsafe node is defined as* strongly unsafe *if none of its nearest neighbors is safe. An unsafe, but not strongly unsafe, node is called an* ordinarily unsafe *node.*

An ordinarily unsafe node has at least one safe nearest neighbor. Apparently, it is more difficult to find a "good" next node for a message to advance at a strongly unsafe node. Consequently, a node can be in one of the four states: *safe, ordinarily unsafe, strongly unsafe,* and *faulty.* The fundamental idea of the routing strategy is to avoid forwarding messages to a more severely unsafe node, if possible, to avoid potential pitfalls. The algorithm *Iden_Unsafe* can be easily modified to identify strongly unsafe nodes.

Each node in the hypercube contains four lists, namely *FAULT, SAFE, UNSAFE,* and *S_UNSAFE,* which maintain its faulty, safe, ordinarily unsafe, and strongly unsafe nearest neighbors, respectively. Let *sour* and *dest* denote the source and destination nodes of a message respectively.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

Also let *curr* represent the current node label of the message. To facilitate the following description of the routing algorithm, we define $d_i(curr, dest)$ as the dimension of the ith smallest bit in which *curr* and *dest* differ. Also let $s_i(curr, dest)$ denote the dimension of the ith smallest bit in which *curr* and *dest* are identical. A routing algorithm called *ROUTE* is presented in Fig. 5. Basically, *ROUTE* attempts to route a message via a shortest path whenever possible. Also it always chooses a node with highest safeness to forward the message. If there exists, on a shortest path to the destination, a safe next node, *ROUTE* would choose to route the message via this node. If there exists no such safe node, then *ROUTE* chooses an ordinarily unsafe next node on a shortest path. If the aforementioned next node does not exist, then *ROUTE* chooses a strongly unsafe next node on a shortest path, either when the current node *curr* is strongly unsafe itself or when *curr* is only two hops from the destination *dest*. In case this situation does not exist, the message will be derouted to a safe neighbor if *curr* is ordinarily unsafe or to an ordinarily unsafe neighbor if *curr* is strongly unsafe. Obviously, such deroute is along some dimension $s_i(curr, dest)$. Actual coding of *ROUTE* should arrange the lists *SAFE*, *UNSAFE*, *S_UNSAFE*, and *FAULT* in an ascending order of dimensions. Since the lengths of *SAFE*, *UNSAFE*, *S_UNSAFE*, and *FAULT* lists add up to n for an n-cube, the complexity of *ROUTE* is $O(n)$ with this arrangement. Also $d_i(curr, dest)$ and $s_i(curr, dest)$ can be selected in an ascending order. For example, the faulty hypercube of Fig. 3 is shown in Fig. 6. Node S_1 = (1011) is a safe node. Consider a message is to be sent from S_1 to the ordinarily unsafe node D = (0000). *ROUTE* first chooses safe node (1001) to forward the message. The message is then routed to D via another safe node (0001) through a shortest path of length 3. Now consider sending a message from an ordinarily unsafe node S_2 = (0110) to D. Since all shortest paths to D are blocked by faulty nodes, the message is first derouted to a safe neighbor (0111). It is then routed to D from (0111) via a minimum path as shown in Fig. 6. Consider the case where a message is sent from a strongly unsafe node S_3 = (1110) to node D. Since there exists ordinarily unsafe neighbor on a shortest path to D, *ROUTE* chooses to use the next node (1100). At node (1100), all shortest paths are blocked by faults, the message is derouted to a safe neighbor (1101). It is then routed as shown in Fig. 6. Note that both derouted paths are minimum although they are not shortest. Shortest paths of lengths equal to $H(S_2, D)$ and $H(S_3, D)$, respectively, do not exist for the example shown.

```
Algorithm ROUTE
begin
  l := H(curr,dest);
  if (l = 0)
    Deliver the message to the local node;
  if (∃ a neighbor of curr connected by link of dimension d_i(curr, dest), i ∈ (1, l), in SAFE list)
    then begin
          Route message via link of dimension d_i(curr, dest);
          exit;
    end
  if (∃ a neighbor of curr connected by link of dimension d_i(curr, dest), i ∈ (1, l), in UNSAFE list)
    then begin
          Route message via link of dimension d_i(curr, dest);
          exit;
    end
  if (curr is strongly unsafe) or (l ≤ 2)
      if (∃ a neighbor of curr connected by link of dimension d_i(curr, dest), i ∈ (1, l), in
          S_UNSAFE list)
        then begin
              Route message via link of dimension d_i(curr, dest);
              exit;
        end
  if (∃ a neighbor of curr connected by link of dimension s_i(curr, dest), i ∈ (1, n − l), in SAFE list)
    then begin
          Route message via link of dimension s_i(curr, dest);
          exit;
    end
  if (∃ a neighbor of curr connected by link of dimension s_i(curr, dest), i ∈ (1, n − l), in
      UNSAFE list)
    then begin
          Route message via link of dimension s_i(curr, dest);
          exit;
    end
end
```

Fig. 5. Algorithm *ROUTE* for message routing in a hypercube with unsafe labels.

According to Theorem 1, there exists, between a safe node and any nonfaulty node, at least one feasible minimum path of length equal to the Hamming distance between them.

THEOREM 5. *If any of the source and destination nodes of a message is safe,* ROUTE *will route the message through a feasible, minimum path of length equal to the Hamming distance between the source and destination nodes.*

PROOF. Two cases are considered.

Case 1: The source node *sour* is a safe node.

Initially, we have *curr* = *sour*. If $H(sour, dest) \geq 3$, there always exists one safe neighbor of *sour* on a shortest path to *dest*, to which *ROUTE* forwards the message since *curr* is safe. The message will subsequently be forwarded by *ROUTE* in a similar fashion at the succeeding nodes. Eventually it will reach a safe node *curr* for which $H(curr, dest) = 2$. As *curr* is a safe node, the two nearest neighbors common with *dest* cannot be both faulty. *ROUTE* selects a nonfaulty one to forward the message; the message is then passed to *dest* in the last hop. Obviously, the length of this path is equal to the Hamming distance between *sour* and *dest*.

Similarly, the theorem is true for the case where $H(sour, dest) = 2$ or $H(sour, dest) = 1$.

Case 2: The destination node *dest* is a safe node and *sour* is not safe.

Observe that the $H(sour, dest)$ neighbors of *sour* along dimensions in which *sour* and *dest* differ cannot be all either faulty or strongly unsafe. This can be argued as follows. Consider the case where $H(sour, dest) \geq 3$.

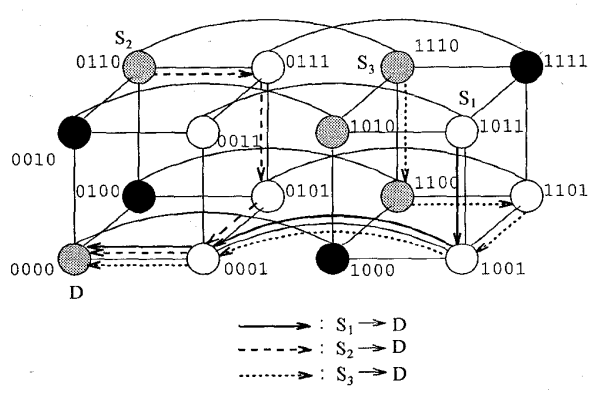


Fig. 6. Routing examples for $S_1 \rightarrow D$, $S_2 \rightarrow D$, and $S_3 \rightarrow D$.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

From previous discussion in case 1, we know that there exists a safe node *curr* on a feasible minimum path from *dest* to *sour* for which $H(sour,curr) = 2$, since *dest* is a safe node. This implies that at least one of the two common neighbors of *curr* and *sour*, which are also on some minimum paths from *sour* to *dest*, must be neither faulty nor strongly unsafe as strongly unsafe node cannot have a safe neighbor, *curr* in this case. As a result, *ROUTE*, at *sour*, can find a fault-free next node, which is not strongly unsafe, on a shortest path to *dest* to forward the message. By repeating this argument, the message will eventually reach a fault-free node *curr*, that is not strongly unsafe and $H(curr, dest) = 2$. As *dest* is a safe node, the two nearest neighbors common with *dest* cannot be both faulty. *ROUTE* selects a fault-free one to forward the message; the message is then passed to *dest* in the last hop. Obviously, the length of this path is equal to the Hamming distance between *sour* and *dest*. Similarly, the theorem is true for the case where $H(sour, dest) = 2$ or $H(sour, dest) = 1$. □

THEOREM 6. *For a message sent from an ordinarily unsafe source node to any fault-free destination node that is not safe,* ROUTE *routes it via a path of length no greater than two plus the Hamming distance between the source and destination nodes.*

PROOF. In this case, *ROUTE* forwards the message to a nearest neighbor, which is safe, or ordinarily unsafe if there are no safe ones, on a shortest path to *dest*. It will eventually lead to a shortest path from *sour* to *dest* if it continues to find such neighbors. However, if the neighbors on all shortest paths to *dest* are either faulty or strongly unsafe at some fault-free intermediate node *curr* which apparently is not strongly unsafe, *ROUTE* deroutes the message to a safe neighbor if $H(curr, dest) \geq 3$. Since *curr* is ordinarily safe, such safe neighbor always exists. As described previously, the message will be routed via a shortest path hereafter. This derouting of the message increases the path length by 2. In case of $H(curr, dest) = 2$, if there exists a fault-free neighbor, whether strongly unsafe or not, on a shortest path to *dest*, *ROUTE* uses such node as the next one to forward the message to *dest*. If these two neighbors are both faulty, a deroute similar to the one described earlier is applied, and thus increases the path length by 2. Hence, we prove the theorem. □

Derouting is taken to avoid possible requirement of backtracking, that may render a longer path after entering a strongly unsafe node. Finally, consider the case where the source node is strongly unsafe and the destination node is not safe. We need the following theorem for further discussion.

THEOREM 7. If an n-cube is not fully unsafe, then every strongly unsafe node has at least one ordinarily unsafe neighbor.

PROOF. Assume A is a strongly unsafe node in the n-cube. Suppose k neighbors of A are faulty. Note that k cannot be equal to n as the n-cube is not fully unsafe, that is, $k \leq n - 1$. Hence, the other $n - k$, $n - k \geq 1$, neighbors of A are unsafe. Now assume that all these $n - k$ unsafe neighbors of A are strongly unsafe. Let N_f and N_s represent the sets of the k faulty neighbors and the $n - k$ strongly unsafe neighbors of A, respectively. Now examine any nonfaulty node B which is of distance 2 from A. If the two common neighbors of A and B are both in N_f, B must be unsafe. On the other hand, if any one of these neighbors is in N_s, B must also be unsafe by Definition 2. In other words, all the nodes of distance 2 from A would be either faulty or unsafe. Note that this situation is similar to that encountered in the argument of Theorem 3 for the case of $k = n$ after the first iteration of labeling unsafe nodes. It would then lead to a fully unsafe n-cube. This, however, contradicts our assumption in the statement. Hence, it is shown that the unsafe neighbors of A cannot be all strongly unsafe, and the theorem follows. □

Theorem 7 implies that a strongly unsafe node can determine whether the hypercube is fully unsafe by examining the status of its neighbors. If and only if all of the neighbors of a strongly unsafe node are either faulty or strongly unsafe, then the hypercube is fully unsafe. We now return to the case of routing under consideration.

THEOREM 8. *For a message sent from a strongly unsafe source node to any fault-free destination node that is not safe,* ROUTE *routes it via a path of length no greater than four plus the Hamming distance between the source and destination nodes, if the hypercube is not fully unsafe.*

PROOF. *ROUTE* forwards a message via a shortest path to the destination whenever there exists an ordinarily unsafe neighbor, or a strongly unsafe neighbor in case no ordinarily unsafe ones exist, on such path. It will eventually lead to a shortest path from *sour* to *dest* if it continues to find such neighbors. If it turns out, in the worst case, that all the neighbors on the shortest paths are faulty, *ROUTE* then deroutes the message to an ordinarily unsafe neighbor. As shown in Theorem 7, such ordinarily unsafe node always exists since the hypercube is not fully unsafe. This derouting increases the path length by 2. By Theorem 6, one can easily obtain that the length of a message path established by *ROUTE* is four plus the Hamming distance between the source and destination nodes in the worst case. □

From the above discussion, we can draw the conclusion that, if an n-cube is not fully unsafe, then *ROUTE* routes a message via a path of length no more than the Hamming distance between the source and destination nodes plus four. *ROUTE* may still be able to route a message when the hypercube is fully unsafe although the possibility is not guaranteed. Also derouting may be required by *ROUTE* even though shortest paths between the end nodes of a message exist. Our observation shows that there exists no strongly unsafe node in an n-cube with no more than $n - 1$ faults. In this case, the length of a path established by *ROUTE* is bounded by the Hamming distance between the source and destination nodes plus two [18].

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

4 Deadlock-Freeness

Deadlock freeness is an essential and desirable property in an interconnection network. Deadlocked situations cause a waste of communication resources, and requires special activities to restore the system back to normal operation. Since deadlock freeness is not specifically taken into consideration in the previous routing algorithm, additional mechanisms are required to maintain such property. To facilitate the following discussion, we assume that the underlying switching technique used in the hypercube is the popular wormhole routing with virtual channels [9].

A well-known method for enforcing deadlock freeness in a resource sharing environment is to assign a set of priorities to the shared resources and to restrict the usage of the resources in certain order of the priority classes. The length of a possible longest path determines the number of virtual channels a hypercube requires in order to achieve a deadlock-free implementation of the algorithm *ROUTE*. Note that the longest distance traversed by a message, using *ROUTE*, is bounded by $n + 3$ in an n-cube which is not fully unsafe. This is argued as follows. Consider the case in which a message is sent from a strongly unsafe node to a node that is not safe and is of distance n apart in an n-cube. Since the n-cube is not fully unsafe, the source node must have an ordinarily unsafe neighbor by Theorem 7. Obviously this neighbor is on a shortest path to the destination as the distance between the source and destination nodes is n. Therefore, the path length for the message can be no longer than $n + 2$ in this case. In other words, a path length of $n + 4$ does not occur. However, a path length of $n + 3$ can possibly occur for the case where the source node is strongly unsafe and the destination is of distance $n - 1$ apart. Hence, each physical channel can be divided into $n + 3$ virtual channels numbered from 1 to $n + 3$. A physical channel, in this case, corresponds to a unique physical link. The $n + 3$ classes of virtual channels are used in an increasing order in an n-cube. A class-1 virtual channel is allocated for the first hop in the routing of a message. The class of virtual channel used for subsequent forwarding of a message is incremented by one for each additional hop. The algorithm *ROUTE* is obviously deadlock-free with this implementation.

4.1 Constant-Number Resources

One of the major disadvantages of the previous deadlock-free implementation is that the number of required virtual channels is dependent on the size of the hypercube under consideration, thereby rendering system design unscalable. In the following, we present another routing algorithm for which each physical channel consists of a constant number of virtual channels, regardless of the size of the hypercube, while the property of deadlock freeness is maintained.

A physical channel of any dimension i is called *up-transition* ($0 \rightarrow 1$) if it is incident out of a node T with $t_i = 0$, i.e., it is from a node with ith bit being zero to the neighbor with ith bit being one. We can define a *down-transition* ($1 \rightarrow 0$) physical channel vice versa. Consider that each physical channel is divided into five virtual channels. The virtual channels in a hypercube are classified in five virtual networks, denoted by VN_i, $1 \le i \le 5$, where i is called the *network number* of the virtual channels in VN_i. Each virtual network contains a virtual channel corresponding to each physical channel. A virtual channel is called an *up virtual channel* if its corresponding physical channel is up-transition. Similarly, we can define a *down virtual channel*. Furthermore, each VN_i, $1 \le i \le 5$, is partitioned into two virtual subnetworks, denoted by VN_i^u and VN_i^d, which consist of up virtual channels and down virtual channels within VN_i, respectively.

In Fig. 7, we present another fault-tolerant routing algorithm called *ROUTE_C*. In the algorithm, $d^u(curr, dest)$ and $d^d(curr, dest)$ denote the sets of up-transition and down-transition dimensions in which the nodes *curr* and *dest* differ, respectively. The notation $s^u(curr, dest)$ denotes the set of dimensions in which *curr* and *dest* are identical and are both zero. Similarly, $s^d(curr, dest)$ denotes the sets of dimensions in which *curr* and *dest* are identical and are both one. The variable v indicates the network number of the virtual network to be used, and (v, i) represents the virtual channel of the network VN_v along dimension i at the current node. The value of v is set to one at the source node. Other related notations are adopted from the previous discussion. With algorithm *ROUTE_C*, we route a message using the up virtual subnetwork first and then switch to a down virtual subnetwork when no up virtual channels are available

```
Algorithm ROUTE_C
begin
  l := H(curr,dest);
  Set v← the incoming virtual network number; { v is set to 1 at the source node}
  if (currently using up-transition channel)
        call FORWARD(d^u(curr, dest), d^d(curr, dest));
  else  call FORWARD(d^d(curr, dest), d^u(curr, dest));
  if (the message has not been forwarded)   {Deroute takes place}
       if (currently using up-transition channel)
             call FORWARD(s^u(curr, dest), s^d(curr, dest));
       else  call FORWARD(s^d(curr, dest), s^u(curr, dest));
end

Function FORWARD(dim_set1, dim_set2)
begin
  if (∃ a neighbor of curr along dimension i ∈ dim_set1 which is in SAFE list)  {
       Route the message via virtual channel (v, i);
       exit; }
  if (∃ a neighbor of curr along dimension i ∈ dim_set2 which is in SAFE list)  {
       if (currently using down-transition channel)
            v← v + 1;
       Route the message via virtual channel (v, i);
       exit; }
  if (∃ a neighbor of curr along dimension i ∈ dim_set1 which is in UNSAFE list)  {
       Route the message via virtual channel (v, i);
       exit; }
  if (∃ a neighbor of curr along dimension i ∈ dim_set2 which is in UNSAFE list)  {
       if (currently using down-transition channel)
            if (v = 1 or l ≤ 2)
                 v← v + 1;
            else  exit;
       Route the message via virtual channel (v, i);
       exit; }
  if (∃ a neighbor of curr along dimension i ∈ dim_set1 which is in S_UNSAFE list) and
     (curr is strongly unsafe or l ≤ 2) {
       Route the message via virtual channel (v, i);
       exit; }
  if (∃ a neighbor of curr along dimension i ∈ dim_set2 which is in S_UNSAFE list) and
     (curr is strongly unsafe or l ≤ 2) {
       if (currently using down-transition channel)
            if (l ≤ 2)
                 v← v + 1;
            else  exit;
       Route the message via virtual channel (v, i);
       exit; }
end
```

Fig. 7. Algorithm *ROUTE_C* which requires a constant of five virtual networks.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

within a virtual network. Furthermore, virtual networks are used in an increasing order of their network numbers. Within any virtual subnetwork, messages can be routed in an arbitrary order of dimensions. Note that the sizes of $d^u(curr, dest)$, $d^d(curr, dest)$, $s^u(curr, dest)$, and $s^d(curr, dest)$ add up to n for an n-cube. Recall that the lengths of *SAFE*, *UNSAFE*, *S_UNSAFE*, and *FAULT* lists also add up to n for an n-cube. An analysis of the algorithm yields that the complexity of *ROUTE_C* is $O(n)$.

THEOREM 9. *If a hypercube of any size is not fully unsafe, five virtual networks suffice to guarantee deadlock freeness for ROUTE_C.*

PROOF. First of all, we show that five virtual networks suffice for *ROUTE_C* to route a message. There are three cases to be considered here.

Case 1: The source node is a safe node.

Among the dimensions on shortest paths to the destination, there may be some up transitions. The algorithm continues selecting a safe next node along any such dimension, as long as it exists, to forward the message using up virtual channels in VN_1^u subnetwork. Note that such safe node always exists when there are more than two up-transition dimensions between the current and destination nodes since the current node is a safe one. It then switches to route on VN_1^d to safe nodes along down-transition dimensions on shortest paths to the destination. Similarly, such safe nodes always exist when there are more than two down-transition dimensions between the current and destination nodes. Consequently, there are, in the worst case, at most four hops, two up-transition hops and two down-transition ones, left when it has to switch to VN_2^u at a safe intermediate node, where no down-transition traversal is possible at the node. If there are more than two hops from this node to the destination, there exists at least one up-transition dimension that is on some shortest paths to the destination, and leads to a safe next node. Therefore, the virtual channel belonging to VN_2^u leading to safe neighbors can be subsequently used for advancing. By the same token, VN_2^d can be used to forward the message to safe nodes along shortest paths, if necessary. In summary, virtual network VN_2 is available for forwarding the message to a node within a distance of 2 from the destination, if necessary. Finally, virtual network VN_3 can be used for delivering the message to its destination since the current node is safe and is no more than two hops from the destination. Note that, in this case, the message is routed via only dimensions in which the source and destination differ. Hence a total of three virtual networks suffice for *ROUTE_C* to route a message originating from a safe source node to the destination via a shortest path of length equal to the Hamming distance between the source and destination nodes.

Case 2: The source node is an ordinarily unsafe node.

According to *ROUTE_C*, if there exists any safe neighbor along a dimension on a shortest path to the destination, one such dimension will be traversed first using either virtual channel of VN_1^u, if such up-transition dimension exists, or virtual channel of VN_1^d otherwise. On the other hand, if there exists no safe neighbor on any shortest path, the algorithm would check to see if ordinarily unsafe next nodes exist on shortest paths. If such ordinarily unsafe nodes exist, it would forward the message to one of these nodes along up-transition dimension first using a virtual channel of VN_1^u. This continues until up-transition traversals leading to ordinarily unsafe nodes are exhausted, the algorithm switches to use virtual channels of VN_1^d if there exist ordinarily unsafe next nodes on shortest paths that are on down-transition dimensions. Eventually, either when all shortest paths are blocked due to faulty nodes and/or strongly unsafe nodes, or when VN_1 network cannot be used for further shortest-path routing, the message is derouted to a safe node, which is guaranteed to exist since the current node is ordinarily unsafe. Note that one cannot use virtual channels of VN_1^u after using those of VN_1^d. In this case, if the derouting dimension is down-transition, virtual channels of VN_1^d can still be used. However, if the dimension is up-transition, virtual channels of VN_2^u must be used. Apparently, it is possible that a safe node may exist on a shortest path during the aforementioned procedure. *ROUTE_C* would route the message to such node without derouting. In other words, the algorithm is restricted to consume at most one virtual network, namely VN_1, before the message reaches a safe intermediate node. As described in the previous case, at most three virtual networks are required for the message to reach the destination afterwards. Note that the use of a virtual channel of VN_2^u for derouting, as mentioned earlier, does not consume an extra virtual network for the routing since it is up-transition. Therefore, a total of four virtual networks suffice for *ROUTE_C* to route a message from an ordinarily unsafe node.

Case 3: The source node is a strongly unsafe node.

According to *ROUTE_C*, if there exists any ordinarily unsafe neighbor along a dimension on a shortest path to the destination, one such dimension will be traversed first using either a virtual channel of VN_1^u, if such up-transition dimension exists, or a virtual channel of VN_1^d otherwise.

On the other hand, if there exists no ordinarily unsafe neighbor on any shortest path, the algorithm would check to see if strongly unsafe next nodes exist on shortest paths. If such strongly unsafe nodes exist, it would forward the message to one of these nodes along an up-transition dimension first using a virtual channel of VN_1^u. This continues until up-transition

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

traversals leading to strongly unsafe nodes are exhausted, the algorithm then switches to use virtual channels of VN_1^d if there exist strongly unsafe next nodes on shortest paths that are on down-transition dimensions. Eventually, either when all shortest paths are blocked due to faulty nodes, or when VN_1 network cannot be used for further shortest-path routing, the message is derouted to an ordinarily unsafe node, which is guaranteed to exist according to Theorem 7 since the hypercube is not fully unsafe. In this case, if the derouting dimension is down-transition, a virtual channel of VN_1^d can still be used. However, if the dimension is up-transition, a virtual channel of VN_2^u must be used. Apparently, it is possible that a ordinarily unsafe node may exist on a shortest path during the aforementioned procedure. *ROUTE_C* would route the message to such node without derouting. In other words, the algorithm is restricted to consume at most one virtual network, namely VN_1, before the message reaches an ordinarily unsafe intermediate node. As described in case 2, where the source node is ordinarily unsafe, at most four virtual networks are required for the message to reach the destination afterwards. Note that the use of a virtual channel of VN_2^u for derouting, as mentioned earlier, does not consume an extra virtual network for the routing since it is up-transition. Therefore, a total of five virtual networks suffice for *ROUTE_C* to route a message from a strongly unsafe node.

Finally, we prove that routing by *ROUTE_C* is deadlock-free in the following. Consider the dependency graph of all virtual channels with respect to *ROUTE_C*. Suppose that a cycle exists in the dependency graph. Note that the virtual channels on this cycle cannot all belong to the same virtual subnetwork. This is due to that virtual channels of a virtual subnetwork are all of the same direction (either all up-transition or all down-transition), and thus cannot possibly form a cycle alone. Now, let v_m denote the maximum of network numbers among all the virtual channels on the cycle. Also define virtual channel c_m as follows. Let c_m denote any virtual channel on the cycle with network number v_m, that is down-transition if there exists one such channel, otherwise it is up-transition. Note that virtual networks are used by *ROUTE_C* in an increasing order of their network numbers, and up virtual channels have higher priority over the down ones within a virtual network. Therefore, the virtual channel which c_m waits on must be in the same virtual subnetwork. Continue applying the argument, one can conclude that the cycle will contain only virtual channels within the same virtual subnetwork, considering that the cycle is of finite length. However, this contradicts previous argument. Hence, we prove that a cycle cannot exist in the dependency graph, and thus deadlock cannot exist for *ROUTE_C*.

Hence the theorem is proved. □

The requirement of constant virtual networks in *ROUTE_C* is achieved at the expense that some messages, with small probability, may be routed via a path longer than they should have been using the algorithm *ROUTE*, when the source nodes of them are not safe ones. In the case of a safe source node, a shortest path of length equal to the Hamming distance between the source and the destination is guaranteed.

5 Tolerating Link Failures

Although our mechanism is mainly designed to tolerate node failures. Some foregoing results can be readily extended to handle the case of link failures in addition to node failures. In this case, we have to modify the unsafe-node identification procedure, i.e., *Iden_Unsafe*, as follows. Any fault-free end node of a faulty link are accounted as faulty node while we are identifying unsafe nodes in the hypercube. However, this node is simply treated as an unsafe node after all the other unsafe nodes are identified. The definition for a strongly unsafe node remains the same, considering that a neighbor connected by a faulty link cannot be safe according to the previous identification algorithm. For example, we show a 4-cube in Fig. 8, in which there are three faulty nodes (0010), (0100), and (1000), and a faulty link that connects two fault-free nodes (1010) and (1011). The unsafe nodes are labeled by assuming that both(1010) and (1011) are faulty. Nodes (1010) and (1011) are then marked as strongly unsafe and ordinarily unsafe, respectively.

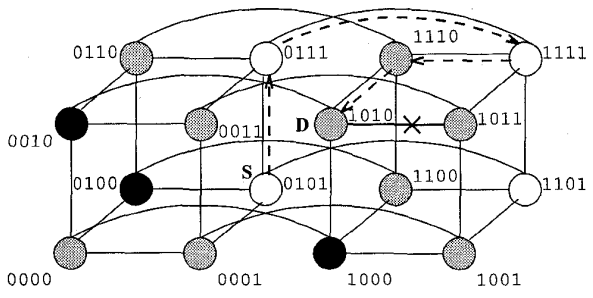


Fig. 8. A 4-dimensional hypercube with three node failures and a link failure.

Consider that node A is safe and node B is two hops away from A, i.e., $H(A, B) = 2$. It is obvious that the two links, that are connected to A and are on shortest paths from A to B, are both fault-free. In addition, the two common neighbors of A and B cannot be both faulty, otherwise A cannot be safe. Now two cases are considered here. If both these neighbors are fault-free, then at least one of the links that connect them to node B must be fault-free, otherwise node A cannot possibly be marked safe as these nodes would be both treated as faulty during the identification of unsafe nodes. Suppose that one of these common neighbors is faulty, then the link that connects the other fault-free node to node B must be fault-free, otherwise A cannot possibly be marked safe. The above arguments lead to that

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.

there exists a fault-free shortest path between nodes A and B. It also implies that the intermediate node on any such path is fault-free, and is not strongly unsafe since A is a safe node. Consequently, it is easy to show that Theorem 1 remains valid, even for any fault-free end node of a faulty link. The argument is similar to the previous proof of Theorem 1, and can be straightforwardly obtained by incorporating the aforementioned result. However, Theorem 4 no longer holds in the presence of link failures.

Algorithm *ROUTE* is modified as follows. No message is routed through faulty links. In addition, when the current node is safe and the distance from the destination is two, *ROUTE* selects a nearest fault-free neighbor on a shortest path to the destination, that is not an end node of any faulty link, to forward the message. As discussed earlier, the existence of such neighbor is guaranteed. Hence Theorem 5 can be readily shown to remain valid. The argument is similar to the proof of Theorem 5 shown earlier. As a result, Theorem 6 is true for the case of link failures under consideration. Furthermore, since the statement of Theorem 7 remains true, Theorem 8 is valid with link failures under consideration. Consider an example in which a message is to be sent from node $S = (0101)$ to node $D = (1010)$ in Fig. 8. The message will be routed as usual. However, when it reaches node (1111), it is node (1110), instead of node (1011), that will be chosen to forward the message to the destination since (1011) is an end node of a faulty link. Similarly, *ROUTE_C* can be extended to maintain the property of deadlock freeness.

6 Conclusion

In this paper, we proposed a fault-tolerant routing strategy which is based on the idea of unsafe nodes for a faulty hypercube system. A set of criteria is presented to identify the unsafe nodes that may cause routing difficulty. In particular, the stringency of the criteria reduces the number of unsafe nodes which are undesirable in the design of routing algorithms. A node in a faulty hypercube can fall in one of four states according to its degree of unsafeness. The routing algorithms presented require each node to maintain only the states of its nearest neighbors. The algorithm *ROUTE* guarantees to route a message via a feasible minimum path of length equal to the Hamming distance between the source and destination nodes of the message if these nodes are not both unsafe. In general, *ROUTE* can always route a message via a feasible path of length no more than four plus the Hamming distance between the source and the destination. The last result is obtained under the assumption that the hypercube under consideration is not fully unsafe. A sufficient condition for this assumption to be true is that the number of faulty nodes is no more than $n - 1$ for an n-cube. Rigorous proofs have been presented. The issue of deadlock freeness is also considered in this paper. More importantly, we proposed another fault-tolerant routing algorithm *ROUTE_C* which requires only a constant of five virtual networks in wormhole routing to ensure the property of deadlock freeness for a hypercube of any size. This scheme makes a hypercube system scalable.

Acknowledgments

The authors would like to thank the referees for their insightful suggestions and comments which serves to improve the paper greatly. Especially, we are indebted to referee E for providing us very knowledgeable references.

This research is supported in part by the National Science Council under grant NSC82-0404-E011-101.

References

[1] D.M. Blough and H. Wang, "Cooperative diagnosis and routing in fault-tolerant multiprocessor systems," *J. Parallel and Distributed Computing*, to appear.

[2] A. Borodin and J.E. Hopcroft, "Routing, merging and sorting on parallel models of computation," *J. Computer and System Sciences*, vol. 30, pp. 13-145, 1985.

[3] M.-S. Chen and K.G. Shin, "Message routing in an injured hypercube," *Proc. Third Conf. on Hypercube Concurrent Computers and Applications*, vol. 1, pp. 312-317, Jan. 1988.

[4] M.-S. Chen and K.G. Shin, "Adaptive fault-tolerant routing in hypercube multicomputers." *IEEE Trans. Computers*, vol. 39, no. 12, pp. 1,406-1,416, Dec. 1990.

[5] M.-S. Chen and K.G. Shin, "Depth-first search approach for fault-tolerant routing in hypercube multicomputers," *IEEE Trans. Parallel and Distributed Systems*, vol. 1, no. 2, pp. 152-159, Apr. 1990.

[6] G.M. Chiu, S. Chalasani, and C.S. Raghavendra, "Flexible routing criteria for circuit-switched hypercubes," *J. Parallel and Distributed Computing*, vol. 22, no. 2, pp. 279-294, Aug. 1994.

[7] G.-M. Chiu and S.P. Wu, "A study on fault-tolerant routing in hypercube systems," Technical Report TR-1993-EE-001, NTIT, Nov. 1993.

[8] P.-J. Chuang and N.-F. Tzeng, "A fast recognition-complete processor allocation strategy for hypercube computers," *IEEE Trans. Computers*, vol. 41, no. 4, pp. 467-479, Apr. 1992.

[9] W.J. Dally and C.L. Seitz, "Deadlock-free message routing in multiprocessor interconnection networks," *IEEE Trans. Computers*, vol. 36, no. 5, pp. 547-553, May 1987.

[10] A.L. DeCegama. *The Technology of Parallel Processing – Parallel Processing Architectures and VLSI Hardware Volume 1*. Prentice Hall, 1989.

[11] P.J. Denning, "Parallel computing and its evolution," *Comm. ACM*, no. 29, pp. 1,163-1,167, Dec. 1986.

[12] J. Duato, "On the design of deadlock-free adaptive routing algorithms for hypercubes: Theoretical aspects," *Proc. Second European Distributed Memory Conf.*, pp. 234-245, 1991.

[13] P.T. Gaughan and S. Yalamanchili, "Adaptive routing protocols for hypercube interconnection networks," *Computer*, vol. 26, no. 5, pp. 12-23, May 1993.

[14] J.M. Gordon and Q.F. Stout, "Hypercube message routing in the presence of faults," *Proc. Third Conf. on Hypercube Concurrent Computers and Applications*, vol. 1, pp. 318-327, Jan. 1988.

[15] T.C. Lee and J.P. Hayes, "A fault-tolerant communication scheme for hypercube computers," *IEEE Trans. Computers*, vol. 41, no. 10, pp. 1,242-1,256, Oct. 1992.

[16] L.M. Ni and P.K. Mckinley, "A survey of wormhole routing techniques in direct networks," *IEEE Computer*, vol. 26, no. 2, pp. 62-76, Feb. 1993.

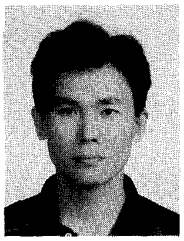
[17] C.L. Seitz, "The cosmic cube," *Comm. ACM*, no. 28, pp. 22-33, July 1985.

[18] S.-B. Tien and C.S. Raghavendra, "Algorithms and bounds for shortest paths and diameter for faulty hypercubes," *IEEE Trans. Parallel and Distributed Systems*, vol. 4, no. 6, pp. 713-718, June 1993.

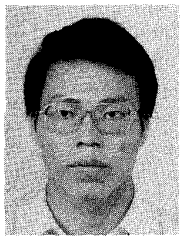
[19] J. Wu, "Safety levels—An efficient mechanism for achieving reliable broadcasting in hypercubes," *IEEE Trans. Computer*, to appear.

[20] J. Wu and E.B. Fernandez, "Reliable broadcasting in faulty hypercube computers," *Proc. 11th Symp. on Reliable Distributed Systems*, pp. 122-129, Oct. 1992.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.



Ge-Ming Chiu received the BS degree in electrical engineering from the National Cheng-Kung University, Taiwan, in 1976, the MS degree in electrical engineering from the Texas Tech University in 1981, and the PhD degree in electrical engineering from the University of Southern California in 1991. In 1991, he joined the faculty of the Department of Electrical Engineering, National Taiwan Institute of Technology, Taipei, Taiwan, where he is currently an associate professor. His current research interests include distributed computing, communication for parallel architecture, and fault-tolerant computing. Dr. Chiu is a member of the IEEE Computer Society.



Shui-Pao Wu received the BS degree in engineering science from the National Cheng-Kung University, Taiwan, in 1988, and the MS degree in electrical engineering from National Taiwan Institute of Technology in 1994. He was with AT&T-TT from 1990 to 1992. He is now an engineer with AT&T-TT. His current research interests include computer networking, telecommunication, and fault-tolerant computing.

Authorized licensed use limited to: Julius-Maximilians-Universitaet Wuerzburg. Downloaded on September 08,2022 at 12:40:47 UTC from IEEE Xplore. Restrictions apply.